APIs verstehen · Kickstart

1 / 10 · Von der Oberfläche zum Server

Dieser Leitfaden erklärt einfache CRUD-APIs, Angular-Services, NgRx Signal Store und Aktionen wie „Export starten“ oder „Projekt zuweisen“. Beispiele beziehen sich auf Angular 22 und NgRx 22; sie beschreiben einen **vorgeschlagenen Serververtrag**, kein bereits vorhandenes Backend.

Vier Bausteine, vier Aufgaben

Baustein	Aufgabe	Beispiel
UI-Komponente	Anzeigen und Benutzerabsicht melden	Speichern geklickt
Feature-Page / Store	Zustand und Ablauf steuern	Ladeindikator, Liste neu laden
API-Service	HTTP und Datenformate kapseln	`projectsApi.create(input)`
Server	Daten speichern und Regeln prüfen	Darf der Benutzer zuweisen?

Datenfluss: Klick → Store-Methode → API-Service → HTTP → Server → Antwort → Store-Zustand → aktualisierte Oberfläche.

Ein Angular-Service ist ein injizierbares TypeScript-Objekt. Unser API-Service enthält HTTP-Aufrufe, aber keinen UI-Zustand. Ein Signal Store hält dagegen die angezeigten Daten, Ladezustände und Fehler. Die Komponente kennt die URL nicht.

Was bedeutet CRUD?

- **Create:** Einen Datensatz anlegen.
- **Read:** Einen oder mehrere Datensätze lesen.
- **Update:** Einen Datensatz ändern.
- **Delete:** Einen Datensatz löschen.

Das Beispielobjekt ist ein Projekt mit `id` und `name`. Zum Einstieg genügt ein API-Service pro fachlicher Ressource und ein Store pro zusammenhängendem Feature. Keinen universellen „kann alles“-Service und keinen Store für jeden Button bauen.

Die zehn Seiten

1. Grundidee und Zuständigkeiten
2. HTTP-Vertrag für CRUD
3. Angular-API-Service
4. Signal Store für CRUD
5. Komponenten anbinden und Zustand verstehen
6. „X an Y zuweisen“ als Ressource oder Aktion
7. Export im Hintergrund starten
8. Einen Hintergrundjob beobachten
9. Jobs in einem Signal Store verwenden
10. Fehler, Wiederholungen und Checkliste

Die Codeschnipsel auf aufeinanderfolgenden Seiten gehören zusammen. Imports werden pro Datei gezeigt; Template-Ausschnitte benötigen eine umgebende Komponente.

Ein einfacher CRUD-Vertrag

2 / 10 · URLs und Antworten zuerst festlegen

Eine API ist eine Vereinbarung zwischen Frontend und Backend. Klärt zuerst URLs, Datenfelder, Fehler und Antworten. Die folgenden Beispiele sind ein bewusst einfacher Ausgangspunkt für kleine Datenmengen.

Zweck	Request	Erfolgreiche Antwort
Projekte lesen	`GET /api/projects`	`200` und `Project[]`
Ein Projekt lesen	`GET /api/projects/p1`	`200` und `Project`
Anlegen	`POST /api/projects` + `{ "name": "Website" }`	`201` und erstelltes `Project`
Teilweise ändern	`PATCH /api/projects/p1` + `{ "name": "Portal" }`	`200` und aktualisiertes `Project`
Löschen	`DELETE /api/projects/p1`	`204`, kein Antwortinhalt

```
{
  "id": "p1",
  "name": "Website"
}
```

GET liest, **POST** erzeugt oder führt einen Befehl aus, **PATCH** ändert gezielte Felder. **PUT** ersetzt typischerweise den Zustand einer adressierten Ressource; Details legt euer Vertrag fest. GET darf keine fachliche Änderung wie einen Exportstart auslösen.

Was gehört in den Vertrag?

- Der Server vergibt IDs; die Anwendung sendet beim Anlegen nur erlaubte Felder.
- Ein neuer Datensatz sollte über `Location: /api/projects/p1` auffindbar sein.
- Nicht gefunden: `404`. Ungültige Eingabe: beispielsweise `422` mit Feldfehlern.
- Nicht angemeldet: `401`. Keine Berechtigung: `403`.
- Bei großen Listen: Filter, Sortierung und Pagination von Anfang an festlegen. Zum Beispiel `{ items: Project[], nextCursor: string | null }` statt eines Arrays.

Dieser Kickstart verwendet absichtlich **keine paginierte Liste**. Eine solche Antwort lässt sich später nicht ohne Anpassung von Service und Store einführen. Das große Architekturhandbuch zeigt bereits einen Cursor-Vertrag.

DTO oder direktes Modell?

Das Beispiel verwendet dieselbe Form für Server und Anwendung. Weicht die API ab, etwa `title` statt `name`, definiert ein `ProjectDto` und mappt es im Service. So bleibt die API-eigene Schreibweise aus Komponenten und Stores heraus. `HttpClient.get<Project>()` liefert TypeScript-Typisierung, **keine Prüfung der Serverantwort zur Laufzeit**. Bei unzuverlässigen externen Daten validieren.

HTTP in einem Service kapseln

3 / 10 · `projects-api.ts`

Einmal in der App-Konfiguration `provideHttpClient()` registrieren. Der Service liefert Observables; er startet keine eigenen Subscriptions und zeigt keine Dialoge.

```
// app.config.ts: bestehende Provider um provideHttpClient() ergänzen
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient } from '@angular/common/http';
export const appConfig: ApplicationConfig = {
  providers: [provideHttpClient()],
};
```

```
// projects-api.ts
import { HttpClient } from '@angular/common/http';
import { Injectable, inject } from '@angular/core';

export interface Project {
  id: string;
  name: string;
}
export interface ProjectInput {
  name: string;
}

@Injectable({ providedIn: 'root' })
export class ProjectsApi {
  private readonly http = inject(HttpClient);
  private readonly base = '/api/projects';
  private url(id: string): string {
    return `${this.base}/${encodeURIComponent(id)}`;
  }
  list() {
    return this.http.get<Project[]>(this.base);
  }
  get(id: string) {
    return this.http.get<Project>(this.url(id));
  }
  create(input: ProjectInput) {
    return this.http.post<Project>(this.base, input);
  }
  update(id: string, input: ProjectInput) {
    return this.http.patch<Project>(this.url(id), input);
  }
  remove(id: string) {
    return this.http.delete<void>(this.url(id));
  }
}
```

Warum kein `subscribe()` hier? Der Aufrufer entscheidet, wann ein Request läuft und wann er endet. Ein HttpClient-Observable führt beim Abonnieren den Request aus; zwei Subscriptions können zwei Requests auslösen.

`/api` nimmt dieselbe Origin oder eine passende Proxy-Konfiguration an. Ein getrenntes Backend braucht eine konfigurierte Basis-URL und passende CORS-Regeln. Zugangsdaten und Authentifizierung zentral konfigurieren, nicht in URLs einbauen. Ein API-Service braucht keinen Signal Store in seinem Konstruktor.

CRUD mit NgRx Signal Store

4 / 10 · `projects-store.ts`

Lese- und Schreibzustand getrennt halten. Der folgende Store lädt die Liste und fasst alle Schreibaktionen in **einer** Ausführungsspur zusammen. Imports und Schreiblogik werden auf Seite 5 ergänzt.

```
import { inject } from '@angular/core';
import { patchState, signalStore, withMethods, withState }
  from '@ngrx/signals';
import { rxMethod } from '@ngrx/signals/rxjs-interop';
import { tapResponse } from '@ngrx/operators';
import { defer, exhaustMap, pipe, switchMap, type Observable } from 'rxjs';
import { ProjectsApi, type Project, type ProjectInput } from './projects-api';

type Write =
  | { kind: 'create'; input: ProjectInput }
  | { kind: 'update'; id: string; input: ProjectInput }
  | { kind: 'remove'; id: string };

export const ProjectsStore = signalStore(
  withState({
    items: [] as Project[], loading: false, writing: false,
    loadError: '', writeError: '',
  }),
  withMethods((store, api = inject(ProjectsApi)) => {
    const load = rxMethod<void>(pipe(
      switchMap(() => defer(() => {
        patchState(store, { loading: true, loadError: '' });
        return api.list().pipe(tapResponse({
          next: items => patchState(store, { items }),
          error: () => patchState(store, {
            loadError: 'Projekte konnten nicht geladen werden.',
          }),
          finalize: () => patchState(store, { loading: false }),
        }));
      })),
    ));
    // Hier folgt die Schreiblogik von Seite 5.
```

`withState` definiert Signals: `store.items()`, `store.loading()` usw. `patchState` ändert Zustand. `rxMethod` verbindet Methodenaufrufe mit RxJS und räumt beim Zerstören des Stores auf. `switchMap` beendet einen älteren Leseaufruf, wenn ein neuer kommt. `tapResponse` verarbeitet Fehler **innerhalb** des Requests, damit die Methode danach weiter nutzbar bleibt.

`defer` setzt den Ladezustand erst beim tatsächlichen Start des inneren Requests. So setzt das Aufräumen des alten Requests den neuen Ladezustand nicht versehentlich zurück. Ein UI-Ladeindikator zeigt Zustand, ersetzt aber keine Fehlerbehandlung.

Schreiben und Komponenten anbinden

5 / 10 · Fortsetzung von `projects-store.ts`

```
    const write = rxMethod<Write>(pipe(
      exhaustMap(command => defer(() => {
        patchState(store, { writing: true, writeError: '' });
        const request: Observable<unknown> = command.kind === 'create'
          ? api.create(command.input)
          : command.kind === 'update'
            ? api.update(command.id, command.input)
            : api.remove(command.id);
        return request.pipe(tapResponse({
          next: () => load(),
          error: () => patchState(store, {
            writeError: 'Änderung fehlgeschlagen. Bitte prüfen.',
          }),
          finalize: () => patchState(store, { writing: false }),
        }));
      })),
    ));
    return {
      reload: () => load(),
      create: (input: ProjectInput) => write({ kind: 'create', input }),
      update: (id: string, input: ProjectInput) =>
        write({ kind: 'update', id, input }),
      remove: (id: string) => write({ kind: 'remove', id }),
    };
  }),
);
```

`exhaustMap` ignoriert weitere Schreibaufrufe, solange einer läuft. Das gilt hier über Create, Update und Delete hinweg. `concatMap` würde stattdessen Aufrufe puffern und nacheinander ausführen; `switchMap` ist für Schreibaktionen meist ungeeignet: Ein beendeter HTTP-Aufruf macht eine Serveränderung nicht rückgängig.

Nach Erfolg lädt der Store neu. Scheitert nur das Neuladen, war das Schreiben trotzdem erfolgreich; `loadError` und `writeError` bleiben deshalb getrennt. Das Formular erst nach bestätigtem Erfolg zurücksetzen, nicht direkt nach `create()`. Die gezeigten Store-Methoden liefern keine Promise für den Abschluss zurück.

Anbindung in der Feature-Page

Den Store in `@Component({ providers: [ProjectsStore], ... })` bereitstellen. In der Klasse: `readonly store = inject(ProjectsStore);` und beispielsweise im Konstruktor `this.store.reload();` aufrufen. Dann im Template:

```
<button
  uiButton
  type="button"
  [disabled]="store.writing()"
  (click)="store.create({ name: 'Neues Projekt' })"
>
  Anlegen
</button>
@if (store.loading()) {
<p role="status">Wird geladen …</p>
} @if (store.writeError()) {
<p role="alert">{{ store.writeError() }}</p>
} @for (project of store.items(); track project.id) {
<p>{{ project.name }}</p>
}
```

`Button` zusätzlich importieren. Ein Feature-Store gehört in den Feature-Scope, ein zustandsloser API-Service darf global sein. NgRx bleibt in der konsumierenden Anwendung und wird keine Laufzeitabhängigkeit der UI-Bibliothek.

„X an Y zuweisen“

6 / 10 · Eine Zuweisung ist oft schon eine Ressource

Nicht jede Benutzeraktion braucht einen Hintergrundjob. Ein Projekt einem Benutzer zuzuweisen kann eine kleine, sofort bestätigte Änderung sein.

Variante A: Beziehung setzen

Wenn jedes Projekt genau eine verantwortliche Person hat:

```
PUT /api/projects/p1/assignee
Content-Type: application/json

{ "userId": "u7" }
```

Antwort beispielsweise `200` mit `{ "projectId": "p1", "userId": "u7" }`. Wiederholt derselbe Request denselben gewünschten Zustand, bleibt das Ergebnis gleich. Entfernen lässt sich die Beziehung per `DELETE /api/projects/p1/assignee`. Bei mehreren Zuweisungen bietet sich eine eigene Collection an.

```
// Zusätzliche Methode in ProjectsApi (Seite 3):
assign(projectId: string, userId: string) {
  const url = `${this.url(projectId)}/assignee`;
  return this.http.put<{ projectId: string; userId: string }>(url, { userId });
}
```

Im Store den `Write`-Typ um `{ kind: 'assign'; id: string; userId: string }` erweitern und die Request-Auswahl um `api.assign(command.id, command.userId)` ergänzen. Die öffentliche Methode meldet nur die fachliche Absicht: `assign(id, userId)`. Nach Erfolg Details bzw. die betroffenen Daten neu laden. Das Beispiel-`Project` zeigt bisher nur `id` und `name`; für sichtbare Zuständigkeiten das Read-Modell ausdrücklich erweitern.

Variante B: Fachlichen Befehl ausführen

Wenn die Zuweisung einen Workflow startet, Genehmigungen prüft oder weitere Arbeiten auslöst, kann ein Befehl klarer sein:

```
POST /api/projects/p1/actions/assign
Idempotency-Key: <stabile-id-dieses-versuchs>
Content-Type: application/json

{ "userId": "u7" }
```

Bei sofortigem Abschluss: `200`/`204`. Bei echter Hintergrundarbeit: `202` mit Job-Referenz wie auf Seite 7. Der Server prüft Zielbenutzer, Berechtigungen und fachliche Regeln. Der Client entscheidet nicht allein, ob eine Zuweisung erlaubt ist.

Einen Export im Hintergrund starten

7 / 10 · Annahme und Fertigstellung sind verschieden

Ein Export kann Minuten dauern. Der HTTP-Request wartet nicht bis zur fertigen Datei, sondern lässt den Server einen dauerhaften Auftrag anlegen.

```
POST /api/exports
Idempotency-Key: 3e8e…
Content-Type: application/json

{ "projectId": "p1", "format": "csv" }
```

```
HTTP/1.1 202 Accepted
Location: /api/jobs/j42
Content-Type: application/json

{ "id": "j42", "status": "queued" }
```

202 heißt: angenommen, noch nicht fertig. Der Server speichert den Auftrag verlässlich und lässt ihn von einem Worker bearbeiten. Ein Timeout im Browser sagt nicht, ob der Auftrag bereits angelegt wurde.

Eine Job-Ressource definiert den Fortschritt

`GET /api/jobs/j42` antwortet beispielsweise:

```
{
  "id": "j42",
  "status": "succeeded",
  "progress": 100,
  "downloadUrl": "/api/exports/e9/download"
}
```

Mögliche Statuswerte: `queued`, `running`, `succeeded`, `failed`, `cancelled`. `progress` ist optional. Eine fehlgeschlagene Verarbeitung kann trotzdem einen HTTP-Status `200` beim erfolgreichen Lesen des Jobs haben: Im Inhalt steht `status: "failed"` und ein kontrollierter Fehlercode.

Serveraufgaben: Job dauerhaft speichern, Eigentümer/Zugriffsrechte prüfen, Idempotenz sicherstellen, Datei geschützt bereitstellen und Aufbewahrungsdauer festlegen. Ein Worker darf beim Neustart keine Aufträge verlieren.

Frontendaufgaben: „Wird gestartet“ von „Export läuft“ und „Fertig“ unterscheiden. Job-ID merken und Status abfragen. Der Job kann weiterlaufen, wenn der Benutzer die Seite verlässt. Für den tatsächlichen Serverabbruch wäre ein eigener, ausdrücklich unterstützter Cancel-Endpunkt erforderlich.

Jobs in einem API-Service beobachten

8 / 10 · `exports-api.ts`

Polling heißt: Den Status regelmäßig erneut lesen. Dieses Beispiel fragt seriell ab; es startet keinen zweiten Statusrequest, bevor der erste beantwortet wurde.

```
import { HttpClient } from '@angular/common/http';
import { Injectable, inject } from '@angular/core';
import { EMPTY, expand, switchMap, timeout, timer } from 'rxjs';

export interface ExportInput {
  projectId: string;
  format: 'csv';
}
export interface Job {
  id: string;
  status: 'queued' | 'running' | 'succeeded' | 'failed' | 'cancelled';
  progress?: number;
  downloadUrl?: string;
  errorCode?: string;
}
export function terminal(job: Job): boolean {
  return ['succeeded', 'failed', 'cancelled'].includes(job.status);
}

@Injectable({ providedIn: 'root' })
export class ExportsApi {
  private readonly http = inject(HttpClient);
  start(input: ExportInput, requestId: string) {
    return this.http
      .post<Job>('/api/exports', input, {
        headers: { 'Idempotency-Key': requestId },
      })
      .pipe(timeout(10000));
  }
  get(id: string) {
    return this.http.get<Job>(`/api/jobs/${encodeURIComponent(id)}`).pipe(timeout(10000));
  }
  watch(id: string) {
    return this.get(id).pipe(
      expand((job) => (terminal(job) ? EMPTY : timer(1500).pipe(switchMap(() => this.get(id))))),
    );
  }
}
```

Jede gelesene Antwort wird ausgegeben, auch der abschließende Status. Danach endet der Stream. Die zehn Sekunden begrenzen **jeweils einen HTTP-Request**, nicht die gesamte Joblaufzeit. Bei einem Requestfehler endet die Beobachtung; der Store auf Seite 9 erlaubt, sie gezielt wieder aufzunehmen.

Ein produktives System sollte Polling-Intervall, Laufzeitgrenze und Wiederaufnahme abstimmen. Bei vielen Clients oder häufigen Statusupdates sind Server-Sent Events oder WebSockets mögliche Alternativen, aber für den Start nicht zwingend nötig. Nur eigene, autorisierte Job-IDs abfragen.

Jobs im Signal Store verwenden

9 / 10 · `exports-store.ts`

```
import { inject } from '@angular/core';
import { patchState, signalStore, withMethods, withState } from '@ngrx/signals';
import { rxMethod } from '@ngrx/signals/rxjs-interop';
import { tapResponse } from '@ngrx/operators';
import { defer, exhaustMap, of, pipe, switchMap, tap } from 'rxjs';
import { ExportsApi, type ExportInput, type Job } from './exports-api';
type Command =
  { kind: 'start'; input: ExportInput; requestId: string } | { kind: 'resume'; id: string };

export const ExportsStore = signalStore(
  withState({ job: null as Job | null, busy: false, error: '' }),
  withMethods((store, api = inject(ExportsApi)) => {
    const run = rxMethod<Command>(
      pipe(
        exhaustMap((command) =>
          defer(() => {
            patchState(store, { busy: true, error: '' });
            if (command.kind === 'start') patchState(store, { job: null });
            const source =
              command.kind === 'start'
                ? api
                    .start(command.input, command.requestId)
                    .pipe(tap((job) => patchState(store, { job })))
                : of({ id: command.id });
            return source.pipe(
              switchMap((job) => api.watch(job.id)),
              tapResponse({
                next: (job) => patchState(store, { job }),
                error: () =>
                  patchState(store, {
                    error: 'Status unklar. Auftrag prüfen oder Beobachtung fortsetzen.',
                  }),
                finalize: () => patchState(store, { busy: false }),
              }),
            );
          }),
        ),
      ),
    );
    return {
      start: (input: ExportInput, requestId: string) => run({ kind: 'start', input, requestId }),
      resume: (id: string) => run({ kind: 'resume', id }),
    };
  }),
);
```

Der Store erlaubt **einen beobachteten Job**. `busy` bleibt bis zum Endstatus oder Beobachtungsfehler aktiv. Weitere Aufrufe werden in dieser Zeit ignoriert. Die Page stellt ihn über `providers: [ExportsStore]` bereit und liest `store.job()?.status`, `store.job()?.progress`, `store.busy()` und `store.error()`. `failed`/`cancelled` gesondert anzeigen; `error` beschreibt hier Transportfehler.

Nach verlorenem Polling `resume(jobId)` verwenden, keinen neuen Export starten. Die Store-Zerstörung stoppt HTTP/Polling, **nicht den Serverjob**. Für Wiederaufnahme nach Navigation/Neuladen die Job-ID passend speichern oder über eine geschützte Jobliste wiederfinden. Download erst bei `succeeded` und vorhandener URL anbieten.

Fehler und Wiederholungen richtig behandeln

10 / 10 · Der kleine Produktionscheck

Gleicher Versuch, gleicher Idempotenzschlüssel

Beim Start einer neuen Benutzerabsicht einmal eine ID erzeugen, zum Beispiel `crypto.randomUUID()`. Eingabe **und** ID zusammen aufbewahren. Für eine Wiederholung nach unklarem Ergebnis denselben Payload und dieselbe ID verwenden. Eine geänderte Eingabe oder ein bewusst neuer Export bekommt eine neue ID. Nicht bei jedem Klick innerhalb eines Retries einen neuen Schlüssel erzeugen.

Der Server muss den Schlüssel für Benutzer/Operation und einen vereinbarten Zeitraum speichern, die ursprüngliche Antwort wiedergeben und abweichende Payloads mit demselben Schlüssel ablehnen. Ein Header allein erzeugt noch keine Idempotenz. Das einfache Create auf Seite 3 bietet diesen Schutz bisher nicht; für sicher wiederholbare Create-Requests muss der Vertrag erweitert werden.

Situation	Sinnvolle Reaktion
GET scheitert vorübergehend	Gezielt wiederholen, ggf. begrenzt mit Backoff
POST-Timeout ohne Job-ID	Ausgang unklar; mit derselben ID wiederholen oder serverseitig nachschlagen
Polling scheitert, Job-ID bekannt	Beobachtung fortsetzen, keinen neuen Auftrag anlegen
Job meldet `failed`	Fachlichen Fehler anzeigen; neuer Versuch nur bewusst
`422`	Feldfehler am Formular anzeigen, Eingabe erhalten
`401` / `403`	Anmeldung bzw. fehlende Berechtigung behandeln
`409` / `412`	Konflikt anzeigen, aktuellen Stand laden und Entscheidung ermöglichen

Zwei Bearbeiter ändern dieselben Daten

Ohne Versionsprüfung kann der letzte Schreibzugriff die Änderung des ersten überschreiben. Für wichtige Daten `ETag` beim Lesen und `If-Match` beim Schreiben vereinbaren. Der Server prüft atomar und antwortet bei veralteter Version mit `412 Precondition Failed`. ETags nicht selbst aus Zahlen erfinden, sofern der Serververtrag diese Abbildung nicht ausdrücklich definiert.

Vor dem produktiven Einsatz klären

- URLs, DTOs, Pagination und Erfolgs-/Fehlerantworten schriftlich festlegen.
- Berechtigungen und Validierung auf dem Server prüfen, nicht nur im UI.
- Bei Cookie-Authentifizierung den passenden CSRF-Schutz berücksichtigen.
- Suchrequests stornieren; Schreibaktionen nicht unbedacht automatisch wiederholen.
- Requests, Fehler, Doppelaufrufe, Konflikte und Polling-Endzustände testen.
- Fehler verständlich anzeigen; keine internen Serverdetails durchreichen.
- Export-Download authentifizieren; Links können ablaufen. Bei Bearer-Auth benötigt ein Download ggf. HttpClient/Blob statt eines einfachen Link-Tags.

Merksatz: Der Service spricht HTTP. Der Store steuert Zustand und Ablauf. Der Server garantiert Datenregeln und dauerhafte Hintergrundarbeit.